

Case Técnico — PM Pleno

Méliuz — In-App Browser

Entrega 2 — Próxima melhoria e PRD

Maio de 2026

Sumário Executivo

A Entrega 1 concluiu que a variante A (Controle) deve ser mantida e que o problema real do In-App Browser não está na ausência de opções de saída para o navegador externo (testadas e descartadas em B e C), mas em fricções de uso que afetam todas as variantes igualmente.

Esta Entrega 2 propõe uma próxima fase de produto estruturada em três componentes complementares:

Componente A — Descoberta dirigida (2-3 semanas):

Investigação aprofundada do segmento de usuários com saída via CONFIG (RPV de R\$ 89,92, o maior observado), para entender quem são, por que têm performance superior e qual o tamanho de mercado endereçável. Não consome esforço de engenharia.

Componente B — Primeira melhoria incremental (autofill nativo):

Implementação de autofill de dados pessoais (CPF, endereço, telefone, e-mail) dentro do In-App Browser, eliminando uma fricção universal sem comprometer o controle do funil. Hipótese primária, sujeita a re-priorização após a descoberta.

Componente paralelo — Spike técnico de Custom Tabs (1 semana de eng):

Investigação técnica curta para entender viabilidade real de resolver login social via Custom Tabs (Android) / SFSafariViewController (iOS). Não é compromisso de implementação — gera input para decisão de roadmap futuro.

Por que esta estrutura?

- Coerente com a recomendação da Entrega 1 (manter A → iterar incrementalmente)
- Descoberta antes de comprometer trimestre de engenharia
- Mantém investigação técnica sobre Custom Tabs sem promessa prematura
- Cada componente entrega valor isolado mesmo se os outros forem cancelados

Diagnóstico e Hipótese

Problema identificado

Os dados da Entrega 1 revelaram três fatos críticos:

1. Saídas externas convertem pior que saídas in-app em todas as variantes — o atrito do In-App Browser é, na verdade, um atrito bom para o funil.
2. Saídas forçadas por login social (utm_term=LOGIN) têm o pior RPV de todas (R\$ 27-36), apontando para uma fricção técnica que existe em todas as variantes.
3. Usuários que saem voluntariamente via CONFIG (R\$ 89,92) representam um segmento de alto valor com comportamento atípico que merece investigação.

Esses fatos sugerem que o caminho de produto NÃO é dar mais rotas de fuga (como B e C tentaram), mas FORTALECER o In-App Browser para que o usuário não precise sair.

Por que priorizar isso agora

- A Entrega 1 mostrou que o controle vence — mas "vencer" não significa "estar otimizado". A oportunidade de melhorar a baseline existe.
- As fricções identificadas (login social, dados pessoais, performance) afetam usuários todos os dias. Quanto mais tempo sem atacar, mais receita perdida.
- Custo de oportunidade de não fazer descoberta: continuar testando variações cegas como B e C, gastando ciclos de eng sem ganho mensurável.

Hipótese de produto

Se reduzirmos fricções pontuais do In-App Browser — como o preenchimento manual repetitivo de dados pessoais — os usuários terão menos motivos para tentar sair para o navegador externo, e mais conversões serão completadas dentro do funil controlado pelo Méliuz.

A descoberta vai refinar essa hipótese identificando QUAL fricção tem maior impacto agregado. A primeira melhoria incremental (autofill) é a hipótese primária por já ter evidência indireta de impacto e baixo risco técnico.

Componente A — Descoberta Dirigida

Objetivo

Mapear o segmento de "usuários de alto valor" identificado na Entrega 1 (saída via CONFIG, RPV R\$ 89,92) e identificar a fricção do In-App Browser com maior impacto agregado em receita.

Perguntas de pesquisa

4. Quem são os usuários que saem via CONFIG? (recorrência, valor histórico, categorias preferidas, frequência de compra)
5. Por que esses usuários têm RPV superior? (ticket maior? conversão maior? categoria específica?)
6. Que fricções esses usuários estão evitando ao sair? (login social? autofill? método de pagamento?)
7. Esses padrões existem na base toda, mesmo em usuários que NÃO usaram a opção CONFIG?
8. Qual fricção do In-App Browser tem mais impacto agregado em receita? (ranking objetivo)

Método

- Análise quantitativa dos dados existentes (visits, transactions, partners)
- Cruzamento com dados de comportamento do app (frequência, retenção, LTV) — fontes adicionais da Méliuz
- Pesquisa qualitativa leve (5-10 entrevistas com usuários do segmento, opcional)
- Auditoria manual de 20-30 lojas para entender que tipos de fricção aparecem no fluxo

Esforço e duração

- Duração: 2-3 semanas
- Esforço: 1 PM full-time, 0,5 analista de dados, opcional pesquisa qualitativa
- Não consome capacidade de engenharia

Entregável

Relatório de descoberta com hipóteses testáveis priorizadas por impacto estimado. Esse relatório é o input que valida (ou re-prioriza) a escolha do Componente B.

Componente B — PRD: Autofill Nativo no In-App Browser

Contexto de negócio

O In-App Browser é o canal principal pelo qual usuários do Méliuz acessam lojas parceiras. A Entrega 1 confirmou que mantê-lo como ambiente padrão maximiza conversão e receita. Porém, a experiência de preencher manualmente dados pessoais a cada compra é uma fricção universal que pode estar causando abandono parcial do funil.

Lojas que oferecem autofill nativo do navegador (via cookies do Chrome/Safari) têm vantagem competitiva sobre o In-App Browser, que começa cada sessão sem dados salvos. Essa assimetria pode ser fechada com uma camada de autofill controlada pelo próprio app Méliuz.

Problema

Usuários que iniciam compra pelo In-App Browser precisam digitar manualmente CPF, endereço, telefone e e-mail em todo checkout. Essa fricção:

- Aumenta tempo até finalização da compra
- Aumenta taxa de abandono em formulários longos
- Pode motivar o usuário a tentar sair para o navegador externo (onde tem dados salvos), reduzindo conversão atribuída

Hipótese

Se oferecermos autofill nativo de dados pessoais dentro do In-App Browser, a fricção de preenchimento manual será reduzida, resultando em aumento de conversão e/ou diminuição do tempo até checkout, sem prejuízo aos guardrails de privacidade e atribuição.

Solução proposta

O app Méliuz mantém um cofre local criptografado com dados pessoais do usuário (autorizados explicitamente). Quando o In-App Browser detecta um formulário em uma loja parceira, oferece preenchimento automático via UI nativa. O usuário confirma quais dados preencher antes da injeção. Funciona apenas em lojas auditadas; demais lojas têm fluxo inalterado.

Escopo do MVP

Dentro do escopo

- Detecção automática de campos em formulários web nas lojas parceiras
- Cofre local criptografado para CPF, e-mail, telefone, endereço completo e CEP
- UI de oferta de autofill (botão/banner discreto)
- Mecanismo de injeção de dados nos campos selecionados pelo usuário
- Funcionamento em iOS e Android
- Cobertura inicial das top 50 lojas parceiras (que representam a maior parte do volume)
- A/B test 50/50 randomizado por customer_id

Fora do escopo (próximas iterações)

- Autofill de dados de pagamento (cartão de crédito) — requer compliance PCI-DSS, fica para fase posterior
- Senhas de login — requer cofre de credenciais separado, decisão para outro ciclo
- Lojas fora do top 50 — após validação do impacto na fase inicial

- Custom Tabs para login social — investigação paralela via Spike (ver seção própria)

Critérios de aceite

- Detecta corretamente campos em pelo menos 95% das top 50 lojas (validado por QA manual)
- Tempo entre detecção do form e exibição da oferta < 500ms
- Tempo de injeção dos dados < 2 segundos após confirmação do usuário
- Dados armazenados localmente, criptografados conforme padrão da LGPD
- Dados sensíveis nunca trafegam em URL ou logs
- Usuário pode editar dados antes da injeção
- Mecanismo de opt-out claramente acessível

Métricas de sucesso

Métrica primária

- RPV (Receita por Visita) na variante de tratamento vs controle — sucesso requer lift $\geq 2\%$ com p-valor < 0,05

Métricas secundárias

- Taxa de conversão: % de visitas que viraram compra
- Ticket médio: valor médio por compra
- Taxa de aceite do autofill: % de visitas em que o autofill foi oferecido e usado
- Tempo até checkout: segundos entre entrada na loja e finalização da compra

Guardrails (não podem piorar)

- Crash rate do app não pode aumentar mais que 5%
- Taxa de atribuição de comissões não pode cair mais que 2%
- Tickets de suporte sobre autofill devem ser menos de 0,5% das visitas
- NPS no fluxo de compra não pode cair

Riscos, dependências e mitigações

Risco	Impacto	Mitigação
Vazamento de dados pessoais	Alto (LGPD, reputação)	Cofre criptografado local, auditoria de segurança antes do lançamento
Detecção falha em lojas customizadas	Médio (UX confusa)	Auditoria das top 50 lojas, fallback silencioso quando detecção falha
Injeção quebra layout da loja	Baixo (1 loja específica)	Lista de exceções configurável via backend
Performance do app cai	Médio (engagement geral)	Detecção lazy (só quando form aparece), benchmarks antes do release

Dependências

- Time mobile (iOS e Android) com capacidade alocada
- Time de dados para implementação de dashboards de monitoramento

- Time de segurança para auditoria de criptografia
- Time jurídico para revisão de LGPD

Plano de Instrumentação

Estratégia

A instrumentação reusa o padrão já existente na Méliuz: parâmetros `mz_*` no JSON de `tracking_url_params` e UTMs na URL de redirecionamento. Isso facilita análise e reduz risco de erros por desvio de padrão.

Novos parâmetros `mz_*`

`mz_test_autofill`

Adicionado ao JSON em `visit_url_metadata.tracking_url_params`. Identifica a variante do A/B test.

```
{ "mz_test_autofill": "a", // ou "b" "mz_redirect": "inapp" }
```

- Valor "a" = controle (sem autofill oferecido)
- Valor "b" = tratamento (autofill nativo disponível)

`mz_autofill_fields`

Novo parâmetro de URL, populado quando o usuário aceita autofill. Lista os tipos de campo preenchidos automaticamente.

```
mz_autofill_fields=cpf,endereco,telefone
```

Novos eventos

Evento	Quando dispara	Propriedades
autofill_offered	App detecta form e exibe oferta de autofill	partner_id, field_types[], variante, customer_id, visit_id, timestamp
autofill_accepted	Usuário clica em "preencher automaticamente"	partner_id, field_types[], variante, customer_id, visit_id, timestamp
autofill_rejected	Usuário ignora ou fecha a oferta	partner_id, field_types[], variante, customer_id, visit_id, timestamp
autofill_completed	Form foi preenchido com sucesso	partner_id, fields_filled[], variante, customer_id, visit_id, latency_ms, timestamp

Exemplo de tracking URL completa

Quando o usuário em variante B aceita autofill e segue para checkout:

```
https://www.meliuz.com.br/redirecionar? utm_source=app& utm_medium=ios& utm_content=partner_page& utm_term=autofill_used& user_id=USER_123& mz_test_autofill=b& mz_autofill_fields=cpf,endereco,telefone& mz_redirect=inapp
```

Validação da instrumentação antes do lançamento

Etapa 1 — Validação em ambiente de staging

- QA executa cenários definidos (cliente preenche form, recusa autofill, autofill em loja específica, etc)
- Time de dados verifica se todos os eventos chegaram corretamente
- Confirma que os parâmetros estão preenchidos com valores corretos

Etapa 2 — Validação por integridade dos dados

- Para cada `visit_id` na variante B, deve existir exatamente 1 evento `autofill_offered` (ou 0 se não houve form)
- Para cada `autofill_accepted`, deve existir um `autofill_offered` correspondente anterior
- Razão `autofill_accepted` / `autofill_offered` deve estar entre 5% e 80% (faixa esperada de adoção)

Etapa 3 — Validação cruzada com fonte alternativa

- Contagem de eventos do app deve bater com logs do backend
- Se app reporta 1000 autofills aceitos e backend recebeu 850 forms preenchidos, há divergência a investigar

Etapa 4 — Rollout canário com observação

- Lançamento inicial para 1% dos usuários por 48 horas
- Validação contínua de métricas e eventos antes de aumentar exposição
- Se qualquer sinal estranho aparecer, pausa imediata para investigação

Plano de Teste e Critérios de Decisão

Tipo de teste

A/B test simples com duas variantes:

- Variante A (controle): In-App Browser sem oferta de autofill
- Variante B (tratamento): In-App Browser com autofill nativo disponível

Divisão 50/50, randomização por customer_id (garante que o mesmo usuário sempre vê a mesma experiência ao longo do teste).

Tamanho de amostra e duração

- Base atual: ~30.000 visitas/dia
- Conversão atual: ~12,9%
- Efeito mínimo detectável (MDE): 1% relativo em conversão
- Poder estatístico: 80%, nível de significância: 5%
- Duração calculada: aproximadamente 3 semanas

Por que 3 semanas:

- Cobre 2 ciclos completos de fim de semana (comportamento de compra varia)
- Permite detectar efeito pequeno com confiança estatística
- Reduz risco de viés por semana atípica

Critérios de decisão pré-definidos

Os critérios abaixo são escritos antes do início do teste, eliminando viés de confirmação na análise dos resultados.

Cenário 1: Sucesso claro

- RPV de B > RPV de A com p-valor < 0,05
- Lift $\geq$ 2% no RPV
- Todos os guardrails dentro do limite

Decisão: rollout para 100% dos usuários em 2 semanas (gradual: 10% $\rightarrow$ 50% $\rightarrow$ 100%).

Cenário 2: Sucesso parcial ou inconclusivo

- RPV de B > RPV de A mas p-valor entre 0,05 e 0,10
- Ou lift positivo mas < 2%
- Guardrails OK

Decisão: estender o teste por mais 2 semanas para ganhar potência estatística. Se continuar inconclusivo, rejeitar implementação e investigar com pesquisa qualitativa o porquê do efeito menor que esperado.

Cenário 3: Sem efeito

- RPV de B aproximadamente igual ao RPV de A (p-valor > 0,10)
- Sem diferença significativa em qualquer métrica secundária

Decisão: rejeitar implementação. Documentar aprendizado e voltar para fase de descoberta com hipótese revisada.

Cenário 4: Efeito negativo ou guardrail estourado

- RPV de B menor que RPV de A
- Ou qualquer guardrail estourou

Decisão: rollback imediato. Análise post-mortem para entender o que aconteceu. Não tentar variações sem entender a raiz do problema.

Quebra para Engenharia

A PRD é dividida em 4 entregáveis ordenados. Cada entregável é independente o suficiente para ter critérios de aceite próprios, e pequeno o suficiente para que o time de engenharia possa estimar com precisão. A engenharia define como quebrar internamente em tarefas técnicas.

Entregável 1: Detecção de campos e infraestrutura de autofill

Tamanho estimado: 2 sprints (~4 semanas)

Escopo

- Capacidade do In-App Browser detectar quando uma loja apresenta formulário
- Identificar tipo de campo (CPF, e-mail, telefone, endereço, CEP)
- Banco local criptografado com dados pessoais do usuário
- API interna para recuperar dados salvos do usuário
- Sem UI ainda — é infraestrutura técnica

CrITÉRIOS de aceite

- Detecta corretamente campos em 95% das top 50 lojas
- Armazenamento criptografado conforme LGPD
- Dados sensíveis nunca trafegam em texto plano

Entregável 2: Interface de oferta de autofill e fluxo de usuário

Tamanho estimado: 1 sprint (~2 semanas)

Escopo

- UI do botão/banner "Preencher automaticamente"
- Modal de seleção de dados (usuário escolhe o que preencher)
- Mecanismo de injeção dos dados nos campos da loja
- Feedback visual (loading, sucesso, erro)
- Funciona em iOS e Android com mesma UX

CrITÉRIOS de aceite

- Tempo de preenchimento total < 2 segundos
- Usuário pode editar dados antes de injetar
- Não preenche campos sensíveis sem confirmação adicional

Entregável 3: Instrumentação e A/B test

Tamanho estimado: 1 sprint (~2 semanas)

Escopo

- Novo parâmetro `mz_test_autofill` no JSON de tracking
- 4 novos eventos: `autofill_offered`, `autofill_accepted`, `autofill_rejected`, `autofill_completed`
- Randomização 50/50 por `customer_id`
- Dashboard de monitoramento das métricas primária, secundárias e guardrails
- Validação em staging antes do lançamento

Critérios de aceite

- Todos os eventos disparam corretamente em 100% dos cenários de QA
- Dashboard atualiza com latência menor que 1 hora
- Mecanismo de rollback testado e funcional

Entregável 4: Lançamento gradual e monitoramento

Tamanho estimado: 4 semanas de calendário

Escopo

- Rollout para 1% por 48h → validação inicial
- Rollout para 10% por 1 semana → validação ampla
- Rollout para 50% por 2 semanas → coleta de dados estatísticos
- Rollout para 100% → se métricas baterem critérios de sucesso
- Análise final com decisão escrita

Critérios de aceite

- Decisão tomada com base em critérios pré-definidos
- Documento de resultado publicado para o time
- Aprendizado registrado no repositório de experimentos

Divisão de responsabilidades

Esta quebra entrega o quê, por quê e critério de aceite. A engenharia define o como, em quantos tickets e em que ordem técnica.

- O PM entrega: escopo, prioridade, critérios de aceite
- A engenharia entrega: arquitetura, divisão técnica, estimativa final
- Ambos validam: viabilidade, riscos, prazos

Iniciativas Paralelas e Próximos Passos

Spike técnico: Custom Tabs para login social

Esta investigação ocorre em paralelo ao desenvolvimento do Componente B e não impacta o roadmap principal. O objetivo é gerar input técnico que ajude a decidir, em ciclos futuros, se e quando atacar o problema de login social.

Escopo do Spike

- Duração: 1 semana de calendário
- Esforço: 1 desenvolvedor sênior (Android ou iOS, ou alternar)
- Output: documento técnico de 3-5 páginas (não código de produção)

Perguntas que o Spike deve responder

9. Atribuição: se implementarmos Custom Tabs, os cookies de atribuição da loja persistem corretamente entre In-App Browser e Custom Tab? Como validar isso na prática?
10. Esforço real: qual o tamanho de equipe e tempo realista para implementação completa, considerando descoberta, desenvolvimento, QA e rollout?
11. Compatibilidade: quais das top 50 lojas têm fluxos de login que funcionariam com Custom Tabs vs. quais teriam problemas conhecidos?

O que NÃO é o Spike

- Não é compromisso de implementação
- Não tem código de produção como saída
- Não bloqueia outras prioridades
- Não é discussão de arquitetura — é descoberta técnica

Próximos passos sugeridos

- Sprint 0 (semana 1): kickoff do Componente A (descoberta) + início do Spike técnico
- Sprints 1-2: descoberta em execução, time mobile inicia Entregável 1
- Sprint 3: descoberta finaliza, valida ou re-prioriza Componente B
- Sprints 4-6: desenvolvimento dos Entregáveis 2 e 3
- Sprints 7-10: rollout gradual e análise

Resumo

Esta Entrega 2 propõe uma abordagem deliberada para a próxima fase do In-App Browser, baseada nos aprendizados da Entrega 1 e nos princípios de descoberta antes de execução e iteração incremental antes de transformação.

Por que esta proposta é defensável

- Coerência com a recomendação anterior: manter A e iterar incrementalmente
- Risco controlado: cada componente entrega valor isolado; nenhum compromete trimestre de engenharia sem dado
- Aprendizado garantido: mesmo no pior cenário, a descoberta gera insight estratégico
- Instrumentação alinhada com o padrão Méliuz: facilita análise e reduz risco de erro
- Critérios de decisão pré-definidos: elimina viés de confirmação

O que se ganha

- Visibilidade clara sobre um segmento de usuários valioso ainda não compreendido
- Primeira melhoria mensurável do In-App Browser desde o teste A/B/C
- Dado técnico sobre uma direção de produto futura (Custom Tabs) sem comprometer ciclo atual